



好，寫一份商業規格書，列出使用流程，功能，需要的資源有哪些，product roadmap

這份商規我會以「零售 × 品牌 AI Data Coop」來寫，讓你可以直接拿去當 pitch / README 的藍本。

一、產品定位與目標

產品名稱（暫定）

Sui Retail Data Coop – 基於 Sui + Walrus 的零售 & 品牌 AI 資料合作平台。

核心目的

- 讓多家零售商把匿名化的一方顧客與銷售數據放進同一個「可驗證的資料池」，由品牌或 AI 服務商付費使用，用於廣告投放優化、需求預測等模型。
- 用 Walrus 提供的可驗證、可程式化資料層（blob 是 Sui object、可驗證可用性、版本）保證「誰貢獻了什麼資料、資料何時被誰用過」在鏈上有完整紀錄。^{[1] [2]}
- 用 Tatum 的 Sui RPC + Walrus API，讓你不用自架基礎建設就可以實作這套機制。

對外一句話 Pitch

「我們把零售『一方數據合作』做成一個可驗證的資料聯盟，讓品牌安心訓練 AI，又讓資料擁有者得到透明的分潤。」

二、目標客群與場景

1. 零售商 / DTC 品牌（Data Provider）

- 擁有大量第一方購買、瀏覽、Loyalty Data，希望變現但不想把 raw data 完全交給第三方平台。
- 已經在做或準備做「Retail Media Network」，使用內部與合作品牌的資料做廣告投放。這類網路被視為零售的新高利潤成長來源。^{[3] [4]}

2. 品牌方 / 廣告主（Data Consumer）

- 需要高品質「零售端」行為資料來做受眾建模、廣告歸因與促銷效果分析。
- 不想被單一 walled garden 鎖死，希望在多個零售夥伴間建立可驗證、可比較的資料合作機制。^[5]

3. AI / 分析服務商 (Model Provider)

- 提供 demand forecasting、推薦引擎、廣告優化模型，需要合法取得多家零售匿名化資料以提升模型品質。
- 希望客戶（零售商 / 品牌）看得到模型使用了哪些 dataset、可以 audit 資料來源，增加信任度。^{[6] [1]}

三、使用流程 (User Flow)

Flow A：零售商上傳資料 & 發行 Dataset 資產

1. 資料準備 (off-chain)

- 零售商在自有系統做去識別化與聚合，產生例如「每日商品銷售彙總」、「客群特徵統計」等 CSV/Parquet。

2. 上傳 Walrus (透過後端 + Tatum Storage API)

- 後端拿到檔案後呼叫 Tatum 的 Walrus/Storage API，把 dataset 上傳為 Walrus blob，回傳 blob-id / object id。^{[2] [1]}

3. Sui 上註冊 dataset

- 使用 Tatum Sui RPC 調用 Move 合約 register_dataset，在 Sui 上鑄造一個 Dataset object：
 - 包含：owner、blob_id、schema_meta（欄位說明）、pricing_model、rev_share_config。
- 回傳 dataset 的 on-chain ID（可視為 dataset NFT）。

4. 在前端的「供應者儀表板」顯示 dataset

- 顯示 dataset 名稱、價格、欄位描述、是否已上架。

Flow B：品牌方購買「使用權」並請求模型訓練

1. 瀏覽資料市集

- 品牌登入前端，瀏覽 dataset 列表（來自鏈上 Dataset objects），可用條件篩選：品類、國家、時間範圍、資料量。

2. 建立「資料方案」訂單

- 選擇若干 dataset，設定用途（例如「預測 Q3 某品類需求」）、使用期間、預算。
- 前端生成一個「Data Campaign」草稿，呼叫 Move 合約 create_campaign：
 - 在鏈上生成 Campaign object，關聯多個 dataset ID，記錄預算與分潤比例。

3. 支付與授權

- 品牌用 SUI 或穩定幣付款，合約把金額鎖在 Campaign object，並給模型服務商一個「dataset 使用憑證」（AccessTicket object）。

4. 模型服務商觸發訓練

- 後端根據 AccessTicket 上列出的 dataset IDs，逐一從 Walrus 讀取 blob（可以透過安全的 server-side pipeline）。
- 模型訓練完成後，產生模型報表與評估結果，打包成 JSON/HTML 上傳 Walrus，得到 report_blob_id。

5. 寫回使用報告與分潤

- 後端透過 Tatum RPC 調用 settle_campaign：
 - 在鏈上 UsageRecord object 中記錄：使用了哪些 dataset、訓練輪數、report_blob_id。
 - 按 rev_share_config 把預算拆給各 dataset owner。

6. 品牌查看結果

- 在前端看到：
 - 模型報告摘要（從 report_blob_id 拉），
 - 使用了哪些 dataset（鏈上記錄），
 - 每個 dataset 對結果的貢獻度（可以先簡化）。

Flow C：資料供應者查看收益與使用紀錄

1. 登入供應者儀表板

- 從鏈上拉該供應者名下的 Dataset objects 與 UsageRecord。

2. 查看 dataset 被用在哪些 campaign、賺了多少

- 每筆使用紀錄對應一個 Walrus usage 報表 blob，可點擊檢視詳細統計（例如被讀取的次數、在模型中的權重）。

四、功能規格（Functional Requirements）

1. On-chain（Sui / Move）模組

主要物件

- Dataset
 - 欄位：id、owner、blob_id (Walrus)、schema_uri、pricing_model（如每次使用固定費用、或按 training 小時數）、rev_share_config。
- Campaign
 - 欄位：id、buyer、model_provider、dataset_ids[]、budget、status（Pending / Active / Settled）。
- AccessTicket
 - 欄位：campaign_id、dataset_ids[]、expiry，授權模型服務商在有效期內存取對應 Walrus blob。
- UsageRecord

- 欄位：campaign_id、dataset_ids[]、usage_stats_hash、report_blob_id、settled_amounts[]。

主要 entry functions (簡化版)

- register_dataset(blob_id, schema_meta, pricing, rev_share)
- update_dataset_meta(dataset_id, ...)
- create_campaign(dataset_ids[], budget, model_provider)
- fund_campaign(campaign_id, payment_asset)
- issue_access_ticket(campaign_id)
- settle_campaign(campaign_id, usage_stats_hash, report_blob_id)

重點：合約不需要知道 dataset 內容，只負責誰擁有什麼、授權給誰、什麼時候用過、如何分錢，blob 實體留在 Walrus。^{[1] [2]}

2. Off-chain 後端服務

- **API Gateway / BFF**
 - 封裝對 Tatum Sui RPC / Walrus API 調用，提供前端簡單 REST/GraphQL 介面。
- **Dataset Pipeline**
 - 接受零售商上傳的檔案，呼叫 Tatum Storage/Walrus API 上傳 blob，拿到 blob_id 後調用 register_dataset。
- **Model Worker (可先 mock)**
 - 接收 AccessTicket，根據 dataset 列表從 Walrus 下載資料，跑簡化的訓練（或 mock 模型），產出報表 JSON，上傳 Walrus，最後呼叫 settle_campaign。
- **Cron / Job**
 - 監控 Campaign 狀態（透過 Tatum RPC 讀事件），決定何時自動結算或關閉過期 Campaign。

3. 前端 (Web dApp)

- 角色切換：Retailer / Brand / Model Provider
- **Retailer 儀表板**
 - 上傳 dataset（其實是把檔案交給後端）、查看 dataset 列表、每個 dataset 的收益與使用紀錄。
- **Brand 儀表板**
 - 探索 dataset、市集列表、建立 campaign、查看模型報告與分潤去向。
- **Model Provider 儀表板**
 - 查看自己收到的 AccessTicket、模型執行狀態、成功/失敗紀錄。
- 錢包整合
 - Sui 錢包 (Sui Wallet / Ethos) 登入、付款、簽署交易。

五、非功能需求 (Non-functional)

- 可驗證性 & 不可竄改
 - 所有 dataset 註冊、campaign 建立、分潤事件皆在 Sui 上紀錄；對應的 raw / aggregated data 則在 Walrus，有可驗證的可用性與版本控制。^{[7] [2] [1]}
- 成本與可擴展性
 - Walrus 為大檔案提供可預測的成本（固定 USD/GB/月），適合長期保留 dataset，與傳統雲儲存成本接近但多了去中心化與可驗證優勢。^{[8] [9]}
- 隱私與合規 (hackathon 版本可簡化)
 - 強調只接收匿名化/聚合後資料；PII 留在零售商系統內。
- 易用性
 - 面向傳統使用者：前端避免「加密貨幣術語」，多用「資料供應者 / 方案 / 資料使用紀錄」等語言來包裝。

六、需要的資源 (人力與技術)

人力角色

- 全端 / 前端工程師
 - 實作 dApp 前端、錢包整合、UI/UX。
- Move / Sui 智能合約工程師
 - 設計與實作 Dataset / Campaign 等合約，熟悉 Sui object 模型。
- 後端工程師
 - 串接 Tatum Sui RPC、Walrus Storage API，做 dataset pipeline、model worker mock、job。
- (可選) 資料科學 / AI 工程師
 - 設計一個簡單但說得過去的模型 demo (例如根據歷史銷售預測未來某品類需求)。

技術與外部服務

- Tatum
 - Sui RPC Endpoint (mainnet/testnet)：發送/讀取交易與事件。^{[10] [11]}
 - Walrus / Storage API：上傳 dataset、報表、usage logs。^{[12] [1]}
- Walrus
 - 作為 dataset blob 的儲存層：大檔案、長期保存、可驗證的可用性。^{[13] [14]}
- Sui 錢包與 SDK
 - TypeScript SDK for Sui + Walrus (如果用官方 Walrus TS SDK)，實作前端交易呼叫流程。^{[15] [1]}
- 雲端 (可選)

- 一個輕量的 server / serverless backend 用來跑 dataset pipeline、model worker。

七、Product Roadmap (以 hackathon → PoC 為主)

Phase 0 : Hackathon MVP (現在~提交)

目標：demo 一個 end-to-end 流程，即使很多東西是簡化/硬編也沒關係。

- 合約
 - register_dataset、create_campaign、fund_campaign、settle_campaign 四個主要 flow。
- Walrus
 - 可以成功把一個 dataset 上傳 Walrus，拿到 blob-id 並寫進 Dataset。
 - 訓練報表 (可以是簡單統計) 上傳 Walrus，寫進 UsageRecord。
- 前端
 - 簡單市集 UI：Retailer 上傳一個 CSV、Brand 選這個 dataset 建立 campaign、付款、按一個「執行模型」按鈕，看到結果與分潤變動。
- 後端
 - Model worker 用一個 toy model (例如算 Top K 暢銷商品) 就好，只要可以證明「有用到 Walrus 上的資料」。

Phase 1 : Alpha (hackathon 後 1-3 個月)

- 支援多 dataset 組合，讓 brand 真正可以選多家零售商的資料。
- 引入簡單的 pricing 模型 (例如按 dataset 數量或資料量分級)。
- 增加 dashboard 的視覺化 (趨勢圖、地區分布、campaign 效果)。
- 與 1-2 家小型電商 / 品牌試點 (即便是沙盒資料)。

Phase 2 : Beta / 商業實驗 (3-12 個月)

- 支援更多資料 schema、標準欄位 (品類、地區、通路)。
- 引入更正式的 AI 模型 (需求預測、促銷模擬等)。
- 與現有 AdTech / CDP / Clean room 合作，作為「可驗證 dataset registry + settlement 層」。

你如果想要，下一步我可以直接幫你「Hackathon MVP」再縮成具體的技術 checklist (合約 function 清單、API 介面、前端頁面草圖)，方便你今晚/明天排開工順序。

✻

1. <https://blog.sui.io/verifiable-ai-data-sui-stack/>
2. <https://www.binance.com/zh-TC/research/projects/walrus>
3. <https://commercediabrands Summit.wbresearch.com/blog/first-party-data-retail-media-success-strategy>

4. <https://www.youtube.com/watch?v=3FUhhvY7Vw8>
5. <https://www.decentriq.com/article/retail-data-partnerships>
6. <https://blog.chainbase.com/from-cloud-to-chain-building-the-next-gen-data-lake-with-walrus>
7. <https://blog.sui.io/celebrating-walrus-one-year-anniversary/>
8. <https://communityone.io/servers/1441975503085895894/the-upside/news/walrus-predictable-pricing-023-gb-mo-2026-05-13/>
9. <https://www.luganodes.com/blog/walrus-decentralized-storage>
10. <https://tatum.io/chain/sui>
11. <https://docs.tatum.io/reference/rpc-sui>
12. <https://tatum.io/blog/tatum-walrus>
13. <https://www.mystenlabs.com/blog/announcing-walrus-a-decentralized-storage-and-data-availability-protocol>
14. <https://www.octobot.cloud/what-is-walrus>
15. <https://sdk.mystenlabs.com/walrus>